

mai. 11, 2026

Construindo domínios ricos

convidado Bruno Joaquim Cassia Dobler Fabio Diniz Francisco Schneider
Gabriel Severo Gabriel kohlrausch Ruan Rodrigues
aline.bianchini@t-systems.com ana.santos@t-systems.com
analista.kuehn@outlook.com anderson.hilario@t-systems.com
anderson.perim@t-systems.com andersongabriel_maria@hotmail.com
anselmogabriel421@gmail.com bmf.pedro@gmail.com
bruna.nery@t-systems.com bruno.destro@t-systems.com
caique.rodrigues@t-systems.com d.silva@t-systems.com
danilosilva2017@hotmail.com desenvolvedor@tidsoftware.com.br
douglas.nishiyama@t-systems.com dpsnbsb79@gmail.com
eduardo.kubota@t-systems.com eudy.amparo@t-systems.com
fabio.marinelli@t-systems.com farley.t.i@hotmail.com
felipeneves089@gmail.com gustavo.ferrazfontes@gmail.com
hewerton.souza@t-systems.com i.parmigiani@t-systems.com
igor.romulo.santos@gmail.com jeanderson.silva@t-systems.com
jl.oliveir@gmail.com jonathan.jimenez@iatec.com karinetrevisann@gmail.com
l.regioli@outlook.com leandro.piqueira@t-systems.com
lucas.p.oliveira@outlook.pt lxsilvareis@gmail.com
marcelohcordeiro@gmail.com marcio@almob.com.br
marcos.ammon@gmail.com moraesguga@gmail.com n.segeti@t-systems.com
nadson.dr11@gmail.com Paulo Scatena rafael.dias@t-systems.com
regis.maioli@t-systems.com reinaldo.rodrigues@t-systems.com
renan-silva@t-systems.com renan.baptista.cabral@gmail.com
rodrigo_7sch@hotmail.com tatidornel@gmail.com
thiago.fernandes@t-systems.com victor.prospero@t-systems.com
victor.santos@t-systems.com victor.souza@t-systems.com
vinicius.bonicio@t-systems.com webdeveloperrm@gmail.com
wesley.boccaletto@t-systems.com wsilva@t-systems.com
fabiano.felgas@gmail.com

Anexos  Construindo domínios ricos

Registros da reunião  Transcrição  Gravação

Resumo

Sessão abordou estratégias de modernização via refatoração iterativa com foco em domínio e desacoplamento de sistemas.

Modernização e refatoração estratégica

A refatoração iterativa é preferível à reescrita total para reduzir passivos técnicos. A priorização de resultados de negócio sobre tecnologias garante entregas parciais e escaláveis.

Domínio, eventos e encapsulamento

Eventos de domínio garantem desacoplamento entre componentes, sendo disparados após a persistência bem-sucedida. Entidades devem encapsular regras de negócio próprias, mantendo estado consistente e independente da infraestrutura.

Agregados, acoplamento e encerramento

Agregados devem ser tratados como unidades únicas, com comunicação estritamente entre raízes. A modularização do monólito precede microsserviços para evitar complexidade accidental e falhas distribuídas.

Próximas etapas

- ☐ [Fabio Diniz] Compartilhar Materiais: Enviar links e recursos sobre Domain Driven Design. Incluir a série DDD e livros mencionados.

Detalhes

- **Entrevista conduzida por Inteligência Artificial:** Farley Souza questiona os participantes sobre a realização de uma entrevista técnica conduzida por

uma Inteligência Artificial, expressando incerteza se essa prática é adequada ou se poderia representar uma irregularidade ([00:00:00](#)).

- **Definição de código legado:** Fabio Diniz define código legado como qualquer sistema que a equipe não compreende totalmente, carecendo de clareza, confiança e contexto, independentemente de possuir testes ou boa estrutura. O palestrante destaca que qualquer código em produção ou escrito há mais de um mês pode ser considerado legado, tornando-se um fardo quando a capacidade da equipe de pagar dívidas técnicas é menor do que a necessidade de criar novas funcionalidades ([00:05:45](#)).
- **Desafios da reescrita total:** Fabio Diniz argumenta que a reescrita completa raramente é a melhor estratégia, pois o software deve ser visto como um passivo e a funcionalidade como um ativo ([00:08:25](#)). O palestrante alerta que a reescrita aumenta o passivo sem entregar novas funcionalidades, frequentemente resultando em falhas ao tentar replicar comportamentos do sistema anterior que não estavam devidamente documentados ([00:09:49](#)) ([00:19:17](#)).
- **Vantagens da refatoração iterativa:** A refatoração contínua é apresentada como a abordagem preferível, permitindo que a equipe isole pequenos contextos, reduza a carga cognitiva e realize entregas iterativas de valor para o negócio ([00:20:43](#)). Essa metodologia facilita testes e validações, além de diminuir o risco de impactos negativos generalizados no sistema ([00:21:52](#)).
- **Cenários aceitáveis para reescrita:** O palestrante esclarece que a reescrita torna-se aceitável apenas em condições específicas, como em projetos que não são rentáveis ou em sistemas de pequeno porte onde o risco associado à reescrita é baixo e o custo de refatoração não compensa ([00:23:08](#)).
- **Manifesto de modernização da Exima:** A cultura da Exima prioriza resultados de negócio sobre resultados tecnológicos, defendendo entregas parciais em vez de projetos de longa duração ([00:24:20](#)). A estratégia foca na migração por componentes e capacidades, responsabilidade federada em vez de controle centralizado, e na busca por escalabilidade e resiliência em vez de soluções temporárias ([00:25:42](#)).
- **Problemas com modelos anêmicos:** Fabio Diniz explica que modelos anêmicos, que separam estado de comportamento, descumprem princípios da orientação a objetos ([00:26:56](#)). Embora intuitivos no início, esses modelos geram duplicação de código, falta de encapsulamento e dificuldades de manutenção a longo prazo ([00:28:19](#)).
- **Diretrizes para refatoração:** O palestrante orienta que não se deve refatorar código que não esteja relacionado à funcionalidade sendo desenvolvida para

evitar introduzir bugs ([00:31:02](#)). Fabio Diniz destaca a regra de 80/20, sugerindo evitar esforços em funcionalidades pouco utilizadas, e alerta sobre o efeito da "janela quebrada", onde aceitar um design ruim conduz a concessões futuras ainda piores ([00:33:31](#)).

- **Desacoplamento de contratos externos:** Para evitar a quebra de contratos ("breaking changes") e efeitos colaterais, é fundamental isolar o domínio da aplicação de dependências externas ([00:34:48](#)). Fabio Diniz recomenda o uso de Objetos de Transferência de Dados, conhecidos como DTOs, para traduzir o domínio interno para o mundo externo, garantindo que alterações no domínio não afetem aplicações consumidoras ([00:38:37](#)).
- **Diferenciação entre Modelo de Entrada e Modelo de Visualização:** O palestrante adverte contra o uso do mesmo objeto para entrada (Input Model) e saída (View Model) de dados, pois isso exige lógicas complexas para lidar com cenários distintos, como a presença de identificadores em edições versus a ausência deles em criações ([00:41:18](#)). A criação de objetos específicos para essas finalidades é recomendada como uma prática mais eficiente ([00:44:38](#)).
- **Encapsulamento do domínio:** Fabio Diniz utiliza o exemplo de finalização de um atendimento veterinário para demonstrar como transferir regras de negócio para dentro do objeto de domínio ([00:46:06](#)). O objetivo é assegurar que o objeto seja responsável por suas próprias operações, aumentando a coesão e protegendo o estado contra alterações externas indevidas ([00:49:01](#)) ([00:51:30](#)).
- **Uso de construtores e estado consistente:** A importância de utilizar construtores para garantir que um objeto seja instanciado apenas em um estado válido é enfatizada, juntamente com o uso de definidores privados para impedir que instâncias externas alterem propriedades de forma não controlada ([00:50:08](#)).
- **Agnosticismo de infraestrutura:** O núcleo da aplicação deve permanecer independente de infraestrutura e escolhas tecnológicas externas ([00:55:08](#)). Respondendo a uma pergunta de Farley Souza, Fabio Diniz esclarece que o ideal é manter as regras de validação separadas dos dados, evitando transformar DTOs em objetos com comportamento, o que facilita a realização de testes ([00:56:28](#)).
- **Validação de Dados em DTOs:** Fabio Diniz explica que erros 400 devem ser retornados quando o usuário realiza operações com dados inconsistentes ou não esperados pela API. Eles enfatizam que regras de domínio não devem ser trazidas para DTOs (Data Transfer Objects), pois isso quebra a separação de

camadas e a tendência é que essas validações cresçam desnecessariamente, dificultando a manutenção ([00:59:22](#)).

- **Conceito de Eventos de Domínio:** Fabio Diniz define eventos de domínio como acontecimentos ocorridos no passado, que devem ser propagados dentro do próprio domínio. Eles ressaltam que a execução de um evento deve ocorrer após a ação realizada, não antes, e que esses eventos operam em memória, frequentemente utilizando a biblioteca MediatR ([01:01:29](#)).
- **Implementação Inicial de Eventos com MediatR:** Fabio Diniz demonstra a implementação de um evento de finalização de consulta, o qual herda de uma interface de notificação. Eles descrevem um manipulador (handler) que utiliza um serviço de notificação para processar o evento, mencionando a boa prática de incluir um token de cancelamento para operações assíncronas ([01:02:38](#)).
- **Problemas de Classes Estáticas e Persistência:** Fabio Diniz alerta que a utilização de classes estáticas para publicação de eventos dificulta a realização de testes unitários. Além disso, eles apontam o risco de publicar eventos antes da persistência dos dados no banco, o que pode causar inconsistência caso a operação de gravação falhe posteriormente ([01:03:56](#)).
- **Publicação Adiada de Eventos:** Para mitigar riscos de persistência, Fabio Diniz propõe a criação de uma interface `IDomainEvent` e uma classe abstrata de entidade. Essa abordagem permite que os eventos sejam armazenados em uma lista privada dentro da entidade e disparados apenas após a confirmação da transação no banco de dados ([01:06:13](#)).
- **Disparo de Eventos após SaveChanges:** Fabio Diniz explica que o disparo dos eventos deve ocorrer apenas após o comando `SaveChanges` ter sido executado com sucesso pelo Entity Framework. Eles detalham o uso do `ChangeTracker` para identificar as entidades alteradas e iterar sobre seus eventos pendentes para publicação ([01:08:45](#)).
- **Desacoplamento através de Eventos de Domínio:** Fabio Diniz destaca que eventos de domínio permitem que componentes se tornem reativos, reduzindo o acoplamento forte entre classes. Eles exemplificam com o cenário de criação de uma fatura, onde o módulo financeiro pode reagir ao evento de "fatura criada" sem que o módulo de faturas precise conhecer os detalhes internos do financeiro ([01:11:26](#)).
- **Simplicidade em Event Handlers:** Fabio Diniz orienta que os manipuladores de eventos devem ser simples e enviar o mínimo de informações necessárias, preferencialmente apenas o identificador (ID). Eles alertam contra a inclusão

excessiva de propriedades nos eventos, o que poderia gerar contratos rígidos e desnecessários com outros sistemas ([01:13:52](#)).

- **Diferenciação de Eventos entre Projetos:** Farley Souza questiona sobre o lançamento de eventos para outros projetos. Fabio Diniz esclarece que, neste contexto inicial, tratam-se de eventos internos de memória, e que a comunicação entre projetos distintos envolverá futuramente eventos de integração, que possuem natureza diferente ([01:15:03](#)).
- **Comunicação entre Subdomínios:** Igor Romulo pergunta sobre a comunicação entre subdomínios dentro do DDD. Fabio Diniz confirma que os eventos de domínio servem justamente para facilitar essa comunicação entre componentes do mesmo software, mantendo o acoplamento baixo ([01:17:19](#)).
- **Eventos de Integração vs Eventos de Domínio:** Fabio Diniz explica que eventos de integração são destinados a sistemas externos e são inerentemente assíncronos, podendo utilizar tecnologias como mensageria (RabbitMQ, Redis, Kafka). Eles reforçam a importância de não expor o mundo interno da aplicação para o externo através de eventos de domínio diretamente ([01:18:05](#)).
- **Processamento Síncrono e Assíncrono no Pipeline:** Márcio das Chagas Ramos questiona se a API aguarda o processamento dos eventos. Fabio Diniz esclarece que o comportamento depende da implementação escolhida, podendo a API aguardar ou não a conclusão do disparo dos eventos após a persistência no banco ([01:21:53](#)).
- **Isolamento e Acoplamento:** Fabio Diniz reforça que o objetivo principal do uso de eventos é evitar que o domínio da aplicação precise conhecer detalhes de infraestrutura ou requisitos de sistemas externos, mantendo a regra de negócio isolada e agnóstica ([01:32:43](#)).
- **Exemplo Prático de Cadastro e Reatividade:** wallace silva descreve sua prática de cadastrar clientes e disparar um evento para que outros serviços consumam a informação. Fabio Diniz valida essa abordagem como correta, pois permite que os sistemas interessados reajam de forma desacoplada ao estado modificado ([01:36:32](#)).
- **Definição de Serviços de Domínio:** Fabio Diniz introduz os serviços de domínio como classes que contêm regras de negócio que não pertencem especificamente a uma entidade única. Eles observam que, enquanto serviços de aplicação orquestram fluxos, serviços de domínio focam na lógica de negócio complexa ([01:37:32](#)).

- **Exemplo Prático de Saque (ATM):** Fabio Diniz ilustra o uso de um serviço de domínio para o saque em um caixa eletrônico (ATM). O serviço encapsula a lógica de cálculo de comissão e a interação com um gateway de pagamento, utilizando a entidade ATM apenas para validar se o saque é permitido ([01:40:15](#)).
- **Regras de Negócio e Cláusulas de Guarda:** Fabio Diniz explica que o domínio deve manter sua coesão, utilizando cláusulas de guarda para validar operações antes de prosseguir. Se uma regra exige consultar um serviço externo, como um gateway de pagamento, ela deve ser encapsulada em um serviço de domínio para não poluir a entidade com dependências de infraestrutura ([01:42:50](#)).
- **Dependências Externas em Serviços de Domínio:** Fabio Diniz justifica a injeção de serviços externos (como um gateway de pagamento) em serviços de domínio. Eles argumentam que isso mantém a entidade pura e testável, movendo a complexidade da integração para uma camada apropriada ([01:45:45](#)).
- **Diferença entre Serviço de Domínio e de Aplicação:** Fabio Diniz reafirma que serviços de domínio processam regras de negócio, enquanto serviços de aplicação, como controladores em MVC, gerenciam o fluxo de execução ([01:48:16](#)).
- **Critérios para Criação de Serviços de Domínio:** Fabio Diniz orienta o uso de serviços de domínio quando a lógica não se encaixa em uma entidade, evitando a duplicação de código e garantindo que o código permaneça expressivo e compreensível ao ser lido futuramente ([01:50:55](#)).
- **Agregados e Consistência de Transação:** Fabio Diniz define agregados como um conjunto de objetos (entidades e objetos de valor) que devem ser tratados como uma unidade em transações de banco de dados. Eles enfatizam que a persistência deve garantir um estado consistente tanto na gravação quanto na recuperação dos dados ([01:53:23](#)).
- **Agregate Root e Comunicação entre Agregados:** Fabio Diniz explica que o Aggregate Root (raiz do agregado) é o responsável por garantir a consistência de todo o conjunto. Eles orientam que a comunicação entre diferentes agregados deve ocorrer sempre através de seus respectivos Roots, jamais acessando entidades internas de outros agregados, para preservar o encapsulamento ([01:55:48](#)).
- **Lógica de Deleção em Agregados:** Fabio Diniz explicou que ao deletar uma raiz de agregado, os objetos filhos também devem ser deletados para manter a coerência do sistema. Entretanto, a deleção de um item dentro de um

pedido não exige a deleção do agregado inteiro, pois um pedido pode existir sem itens durante sua fase inicial de montagem ([01:58:24](#)).

- **Raízes de Agregado e Validação de Construtor:** Fabio Diniz descreveu a implementação de raízes de agregado, que herdam de uma classe base e gerenciam eventos. O agregado de pedido, por exemplo, deve ser instanciado com um estado válido, incluindo ID, cliente, data de criação, total e status, sendo que a lista de itens pode estar vazia, mas deve estar inicializada. Eles enfatizaram que as propriedades devem possuir apenas métodos de leitura (getters) para evitar alterações externas não autorizadas ([01:59:32](#)).
- **Organização de Classes com Partial Classes:** Fabio Diniz observou que, na linguagem C#, as classes parciais (partial classes) permitem a organização de entidades em arquivos separados para melhorar a coesão. Os itens herdam da classe base de entidade, e os construtores são fundamentais para garantir que o objeto inicie em um estado válido, exigindo a definição de ID, descrição e total ([02:00:51](#)).
- **Padrões de Repositório e Comunicação entre Agregados:** Fabio Diniz afirmou que os repositórios devem ser criados para o agregado como um todo, e não para cada entidade individualmente. A comunicação entre diferentes agregados deve ocorrer exclusivamente através das suas raízes, e um excesso de chaves estrangeiras em entidades filhas pode indicar que o agregado está muito grande ([02:01:56](#)).
- **Dimensionamento de Agregados e Boas Práticas:** Fabio Diniz mencionou que um agregado composto por uma única entidade é aceitável. Eles discutiram o princípio YAGNI (do inglês "You Ain't Gonna Need It"), alertando contra a criação de código desnecessário, pois o código funciona como um passivo que exige manutenção ao longo do tempo. Além disso, destacaram que os testes também devem ser tratados como código e mantidos adequadamente ([02:03:24](#)).
- **Encapsulamento e Validade de Estado:** Fabio Diniz explicou que o encapsulamento envolve expor apenas os comportamentos estritamente necessários. Eles alertaram contra a exposição excessiva de contratos em APIs para evitar o acoplamento indesejado entre sistemas e insistiram que objetos nunca devem ser persistidos em um estado inválido, embora estados inválidos transientes durante o processamento sejam aceitáveis ([02:04:43](#)).
- **Encapsulamento versus Isolamento:** Fabio Diniz esclareceu que encapsulamento e isolamento são conceitos distintos; o encapsulamento protege estados e regras imutáveis, enquanto o isolamento separa responsabilidades dentro da aplicação ([02:06:49](#)).

- **Objetos Coesos e Tipos Primitivos:** Fabio Diniz destacou que objetos bem projetados, similarmente a tipos primitivos como DateTime ou String, devem gerenciar seu próprio estado e operações. Eles incentivaram a criação de entidades que sejam coesas e consistentes, encapsulando a lógica dentro dos próprios objetos, em vez de usar tipos primitivos de forma obsessiva ([02:08:04](#)).
- **Identificação e Gerenciamento de Acoplamento:** Farley Souza e Fabio Diniz discutiram a dificuldade de reconhecer o acoplamento no dia a dia. Fabio Diniz sugeriu que o acoplamento, que frequentemente causa falhas em cascata, pode ocorrer em níveis de entidade, componente, módulo ou sistema, e que a percepção desse problema vem da prática constante e da identificação das partes mais críticas do sistema (os 20% do código que geram 80% do trabalho) ([02:10:38](#)) ([02:17:02](#)).
- **Estratégias de Integração e Mensageria:** Fabio Diniz discutiu como os sistemas de mensageria ganharam destaque para integração, pois proporcionam resiliência, diferentemente de integrações baseadas em HTTP, arquivos ou bancos de dados, que frequentemente criam forte acoplamento e exigem que todos os sistemas estejam operacionais simultaneamente ([02:12:43](#)).
- **Complexidade Essencial versus Acidental:** Fabio Diniz explicou que a complexidade essencial provém de necessidades de negócio, enquanto a complexidade acidental é introduzida pelos desenvolvedores. Eles aconselharam cautela ao refatorar métodos de integração legados que atendem aos requisitos de negócio, pois mudanças desnecessárias podem causar instabilidades ([02:13:48](#)).
- **Monólitos versus Microserviços:** Fabio Diniz argumentou que monólitos não são inerentemente ruins e sugeriu refatorar o monólito para um modelo modular antes de considerar uma transição para microserviços. Eles notaram que a migração prematura para microserviços frequentemente introduz problemas graves de integração e complexidade de deploy ([02:14:43](#)) ([02:20:28](#)).
- **Desafios de Deploy e Topologia de Times:** Fabio Diniz compartilhou experiências sobre o gerenciamento de sistemas centrais versus microserviços distribuídos, observando que sistemas distribuídos trazem desafios significativos na coordenação de deploy e depuração. Eles recomendaram o estudo de topologias de times (Team Topologies) para entender a relação entre arquitetura de software e estrutura organizacional ([02:19:13](#)).

- **Enriquecimento do Domínio:** Fabio Diniz orientou que, ao encontrar lógica espalhada pela aplicação, os desenvolvedores devem enriquecer as entidades do domínio movendo essa lógica para dentro dos objetos. Isso reduz a necessidade de objetos comunicarem-se excessivamente, tornando o comportamento intrínseco à entidade ([02:24:06](#)).
- **Recursos Recomendados e Encerramento:** Fabio Diniz forneceu uma lista de materiais, incluindo livros sobre Design Orientado ao Domínio (DDD) e séries de vídeos anteriores, para auxiliar o grupo no entendimento de acoplamento e design de software. A sessão foi encerrada com Fabio Diniz confirmando que esta foi a última aula com o grupo, seguida por despedidas entre Fabio Diniz, Farley Souza e Anderson Perim ([02:28:10](#)).

Revise as anotações do Gemini para checar se estão corretas. [Confira dicas e saiba como o Gemini faz anotações](#)

*Como está a qualidade de **destas observações**? [Responda a uma breve pesquisa](#) para nos dar seu feedback, incluindo o quanto as observações foram úteis para o que você precisa.*